PHASE-II CAPSTONE PROJECT

Self-Healing Microservice Mesh

Team: Tripti (2301010278), Rizpah (2301010219), Raghuveer (2301010268), Ayush (2301010253), Aman (2301010277), Sarthak (2301010216)

GitHub: https://github.com/triptikhandelwal1/self-healing-microservice-mesh

1. Project Overview

Applications are built from many small services (microservices), and any of them can crash, hang or slow down. Fixing this by hand causes downtime. This project is a Go-based control plane that watches a set of deliberately fragile services, detects failures through health probes, restarts failed replicas automatically, and sends traffic only to healthy replicas.

2. Objectives

- Run Services A, B and C as multiple replicas that can be made to fail on demand.
- Detect crashes, hangs, slow responses and errors automatically.
- Restart failed replicas and route traffic only to healthy ones.
- Keep the system at its desired replica count and show detection and recovery times live.

3. Key Features

- Chaos injection: Crash, Hang, Slow and Errors faults can be triggered on any replica.
- Health probing: 1 failed probe = Suspect, 3 failed probes = Unhealthy.
- Auto-restart: The supervisor restarts unhealthy replicas, which rejoin rotation once probes pass.
- Smart routing: A reverse proxy and load balancer skip unhealthy replicas.
- Desired-state scaling: Replica counts can be changed with + and − controls.
- Live dashboard: Replica status, success rate, MTTD, MTTR and an event timeline.

4. Technologies Used

- Go: goroutines for concurrent health checks, channels for probe results, context for timeouts, mutex/atomic for the routing table.
- Go standard library: net/http for services, proxy and dashboard API; os/exec for starting and restarting replicas.
- Web dashboard: browser-based UI for monitoring and fault injection.

5. Implementation

Client requests go through a reverse proxy and load balancer to Services A, B and C (two replicas each, ports 9101 to 9106). The Go control plane has three parts: a Prober that

checks every replica, a Reconciler that compares desired and actual state, and a Supervisor that restarts replicas. A chaos injector breaks replicas on demand.

The control plane runs a reconcile loop on every tick: Observe (collect probe results), Compare (desired vs actual), Act (restart, replace, update routes), Wait (one tick). It acts on current state rather than individual events, as Kubernetes does.

Failure flow: replica fails → first probe fails (Suspect) → third failure (Unhealthy) → removed from routing → supervisor restarts it → probes pass → back in rotation.

6. UI Screenshots

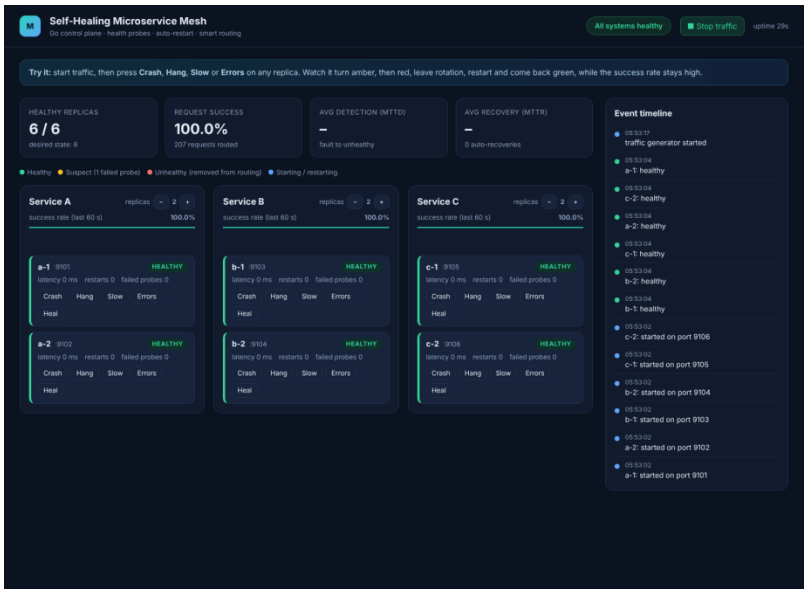


Figure 1: Healthy state

All 6 replicas healthy, 100% request success, desired state 6.

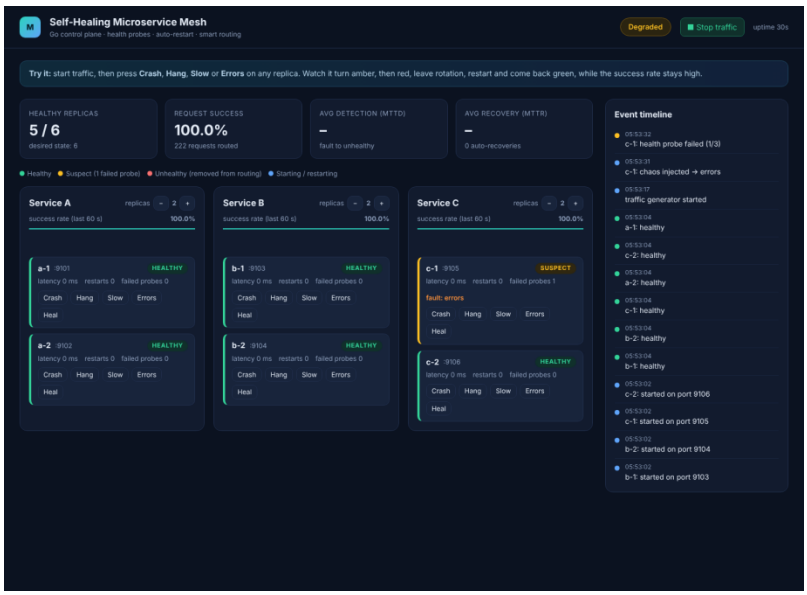


Figure 2: Fault injected

An Errors fault on c-1 makes it Suspect after one failed probe; the header shows Degraded.

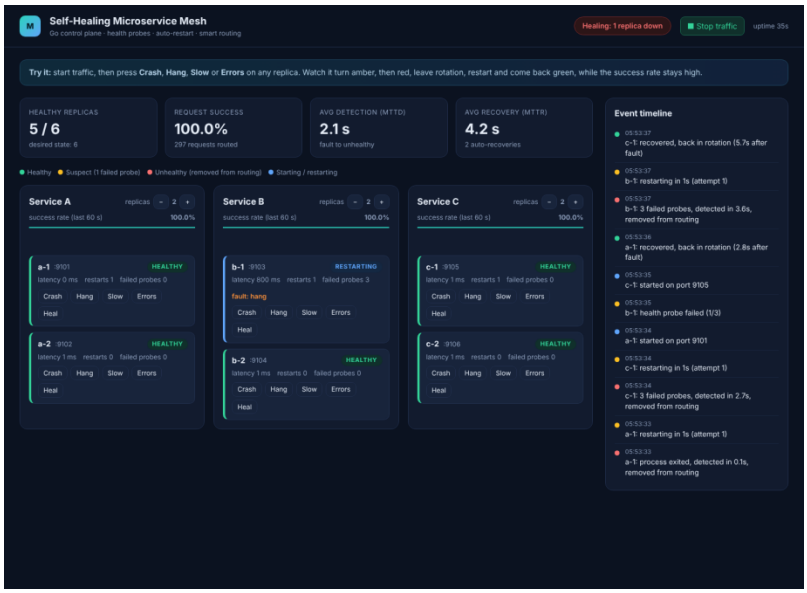


Figure 3: Healing in progress

Failed replicas are removed from routing and restarted; success stays at 100%. MTTD 2.1 s, MTTR 4.2 s.

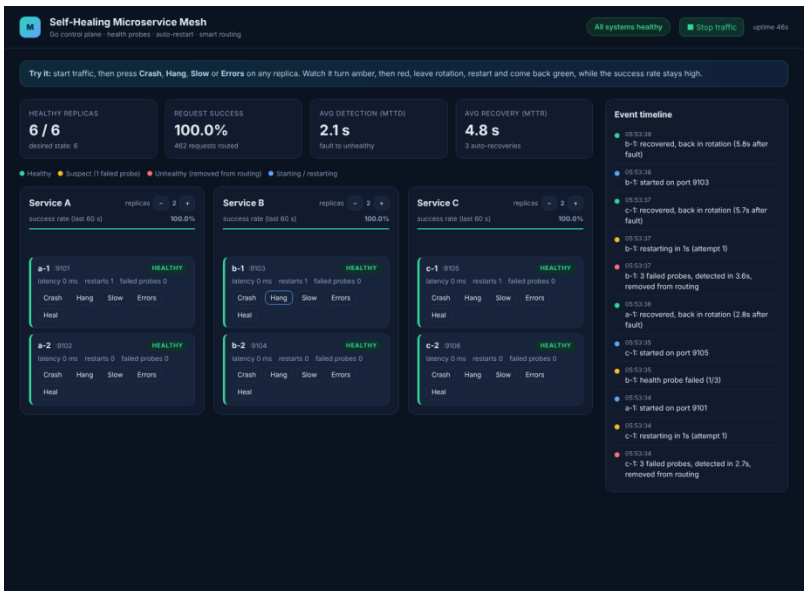


Figure 4: Self-healed

All replicas healthy again after 3 automatic recoveries. MTTD 2.1 s, MTTR 4.8 s.

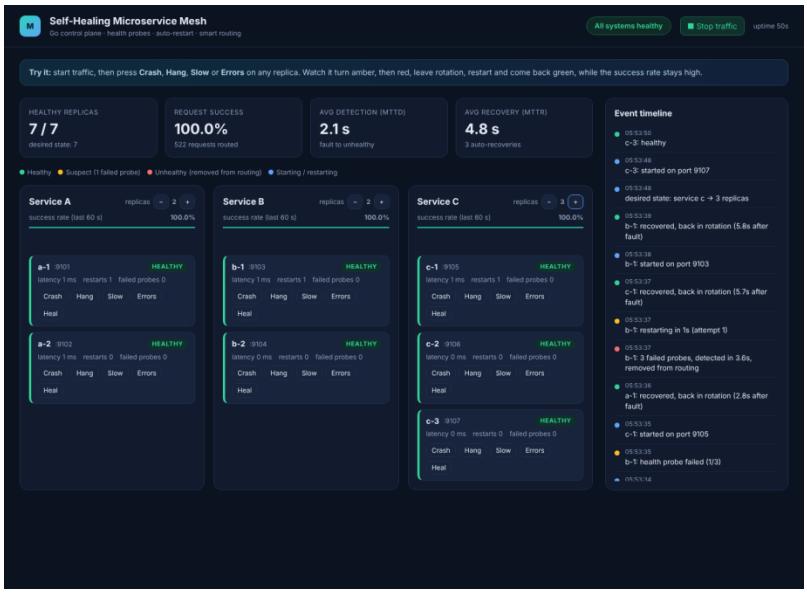


Figure 5: Scaling

Service C scaled from 2 to 3 replicas; c-3 starts on port 9107 and the system shows 7/7 healthy.

7. Conclusion and Future Scope

The system detected faults in about 2 seconds, recovered replicas in under 6 seconds, and kept request success at 100% during faults. Future scope: leader election, auto-scaling and richer dashboard analytics.